FIAP

Computational Thinking using Python — Blueprint Global Solution

Nome do Grupo: Soul Hunter	
Integrantes do Grupo	**RM**
Tárik Moussa Alma	RM: 571411
Giovanni Azevedo	RM: 579428
Fabricio Aquiles Sales da Silva	RM: 570985
Carlos Eduardo Tsucamoto Chiarelli	RM: 569574
Ítalo Neto	RM: 572912

Projeto Integrado: SafraSmart (Versão Final V4)

Disciplina: Computational Thinking using Python

Escopo do Relatório: Descrição Qualitativa, Arquitetura de Módulos e Mapeamento de Funções

1. Descrição da Solução Proposta

O problema central abordado pela Global Solution é a assimetria de informação no campo, onde pequenos e médios agricultores de soja e milho dependem do empirismo devido à dispersão e complexidade de dados climáticos e agronômicos. Para solucionar essa dor, desenvolvemos o SafraSmart, uma plataforma computacional de apoio à tomada de decisão que unifica fontes meteorológicas, imagens de satélite, dados da CONAB e observações locais. O objetivo do sistema é democratizar a agricultura de precisão, mitigando riscos financeiros e aumentando a resiliência climática das lavouras. Suas principais funcionalidades incluem a inserção de dados via interface *offline-first* com captura de GPS e a geração de um score ponderado acompanhado de recomendações probabilísticas de manejo. A linguagem Python é a base de toda a automação e análise de dados do projeto: rotinas consomem múltiplas APIs em cascata com *fallback* automático, enquanto funções calculam estatísticas complexas, como estresse térmico e risco de geada. Além disso, a solução utiliza dicionários Python para cruzar as observações do usuário—processadas via NLP determinístico—com tabelas fenológicas. Através de lógicas condicionais rigorosas, o Python transforma esses dados brutos em ações concretas, auxiliando diretamente o produtor em uma tomada de decisão rápida e quantitativa.

2. Arquitetura do Sistema e Como Executar

O SafraSmart foi concebido de forma modular e escalável, utilizando estruturas nativas do Python para otimizar o processamento local. Abaixo estão as instruções básicas para inicialização do ecossistema:

```
# Instalação das dependências estruturais
pip install -r requirements.txt

# Execução da interface por terminal ou orquestrador local
python menu.py
```

3. Mapeamento Completo dos Módulos e Funções

📁 menu.py — Tela Inicial

Função	O que faz / Requisito Técnico
`limpar_tela()`	Limpa o terminal dinamicamente com suporte multiplataforma (posix/nt).
`carregar_usuario()`	Carrega persistência local com o nome do agricultor saved nas configurações.
`salvar_usuario(nome)`	Armazena permanentemente os metadados do operador para sessões futures.
`perguntar_nome()`	Interfere no fluxo inicial caso seja a primeira rotina do programa.
`main()`	Gere a máquina de estados do menu interativo de 5 opções principais.

Opções do menu estruturado:

- **1. Nova análise (terminal):** Entrada síncrona via input de teclado para geração imediata do diagnóstico.
- **2. Nova análise (web) [BETA]:** Inicializa uma sub-rotina Flask/HTTP local acessível via rede móvel local.
- **3. Executar demo:** Roda pipelines de teste automatizados em tempo real com dados históricos de regiões polo (Sorriso/MT, Cascavel/PR, Uruguaiana/RS).
- **4. Trocar usuário:** Limpa e sobrescreve as configurações de operador.
- **5. Sair:** Encerramento seguro do processo de processamento.

📁 safrasmart/modelos.py — Captura de Variáveis Agronômicas

Função	O que faz / Requisito Técnico
`tipo_cultura()`	Valida a entrada tipada para as culturas suportadas: Soja ou Milho.
`data_plantio()`	Captura e sanitiza strings temporais convertendo para o padrão estrutural dd/MM/aaaa.
`tipo_solo()`	Mapeia o perfil textural do solo informado (arenoso, argiloso ou misto).
`tipo_irrigacao()`	Determina a arquitetura de suprimento hídrico (sequeiro, aspersão ou gotejo).
`prod_anterior()`	Registra a produtividade em sacas da última safra para cálculo de desvio estatístico.
`obs_user()`	Captura logs e notas descritivas em linguagem natural vindas diretamente da lavoura.

📁 safrasmart/clima.py — Barramento de Integração Meteorológica

Função	O que faz / Requisito Técnico
`coordenadas()`	Valida via intervalos matemáticos se a latitude/longitude fornecida pertence ao território nacional.
`buscar_dados_meteo(lat, lon, dias)`	Orquestrador de requisições externas com estratégia de contingência em cascata (fallback sequencial).
`_tentar_archive_api(...)`	Consome o endpoint Open-Meteo Archive buscando séries completas de 365 dias (Prioridade 1).
`_tentar_nasa_api(...)`	Consome a API REST do NASA POWER caso ocorra timeout no servidor principal (Prioridade 2).
`_tentar_forecast_api(...)`	Recorre à API Open-Meteo Forecast coletando modelos preditivos de até 92 dias (Prioridade 3).
`_extrair_dados(dados, fonte)`	Normaliza dicionários JSON heterogêneos vindos das APIs em uma matriz uniforme padrão.
`resumo(dados)`	Agrega visualmente os indicadores climáticos consolidados em formato legível.

📁 safrasmart/indices.py — Mecanismo de Estatística Climática

Função	O que faz / Requisito Técnico
`media_movel_temperatura(dados, janela)`	Aplica o conceito estatístico de média móvel (janela padrão de 7 dias) para identificar tendências térmicas.
`precip_acumulada(dados, inicio, fim)`	Integra os milímetros de chuva contidos no intervalo de tempo parametrizado.
`dias_consecutivos_secos(dados)`	Contabiliza de forma incremental a janela máxima de dias com precipitação zero.
`dias_estresse_termico(dados)`	Monitora anomalias críticas onde a temperatura ambiente excede o teto biológico de 35°C.
`risco_geada(dados)`	Calcula a densidade probabilística de eventos térmicos prejudiciais abaixo de 2°C.
`amplitude_termica(dados)`	Mapeia a variação extrema diária subtraindo as temperaturas mínimas das máximas.
`resumo_climatico(dados)`	Consolida os outputs matemáticos de todas as rotinas em um objeto estruturado de dados.

📁 safrasmart/regras.py — Janelas de Desenvolvimento Fenológico

Este módulo traduz tabelas agronômicas de desenvolvimento em dicionários hash imutáveis estruturados em Python, permitindo o cálculo automatizado com tolerâncias operacionais de 20%.

Tabela de Estágios: Soja

Fase	Intervalo (Dias)	Descrição Fenológica
VE-V3	0 a 20 dias	Germinação e emergência inicial em campo.
V4-V6	20 a 40 dias	Desenvolvimento estrutural e crescimento vegetativo.
R1-R2	40 a 55 dias	Abertura floral e início do ciclo reprodutivo.
R3-R5	55 a 90 dias	Enchimento de vagem e fixação de grãos.
R6-R8	90 a 120 dias	Maturação fisiológica completa e desidratação pré-colheita.

Tabela de Estágios: Milho

Fase	Intervalo (Dias)	Descrição Fenológica
VE-V3	0 a 20 dias	Germinação, emergência e estabelecimento de plântulas.
V6-VT	20 a 60 dias	Crescimento vertical acelerado e pendoamento.
R1	60 a 75 dias	Floração e desenvolvimento do estilo-estigma (espigamento).
R2-R4	75 a 105 dias	Enchimento dos grãos (fases bolha, leitosa e pastosa).
R5-R6	105 a 140 dias	Formação da camada preta e maturação fisiológica do grão.

📁 safrasmart/produtividade.py — Modelagem de Potencial

Ajusta e pondera as curvas de rendimento esperado sacas por hectare considerando solo, irrigação e vigor vegetativo via NDVI. Cruza as coordenadas cartesianas do produtor com dados consolidados da série de relatórios oficiais da **CONAB 2024/25** para derivar linhas de base regionais estáveis por unidade federativa.

📁 safrasmart/ndvi.py — Sensoriamento Remoto via Satélite

Função	O que faz / Requisito Técnico
`autenticar_gee(caminho_chave)`	Estabelece handshake seguro e TLS utilizando chaves Service Account com o Google Earth Engine.
`_executar_ndvi(...)`	Executa reduções espaciais em nuvem nas coleções do satélite Sentinel-2, limpando interferência de nuvens.
`buscar_ndvi(...)`	Invoca o processor remoto acoplado a um wrapper de proteção de tempo limite (timeout de 90s).
`ndvi_por_fase(dados_ndvi, ...)`	Gera a média integrada do índice espectral (valores entre -1 e 1) correspondente a cada janela biológica. Valores acima de 0.6 indicam alta saúde foliar.

📁 safrasmart/observacao.py — Processador NLP Determinístico

Substitui modelos de Inteligência Artificial genéricos por um pipeline de processamento léxico baseado em dicionários de palavras-chave estruturadas com expansão morfológica automática de gênero e número.

Categoria Mapeada	Tokens e Radical de Busca Exemplificados
Pragas Biológicas	lagarta, ferrugem, percevejo, pulgão, lagartas, percevejos
Fitopatologias / Doenças	fungo, oídio, antracnose, mancha, mofo, mofado, fungos
Clima Favorável	chuva boa, úmido, fresco, precipitação adequada
Eventos Climáticos Críticos	geada, seca, granizo, enchente, calor forte, estiagem
Perfis de Solo Degradados	rachado, duro, compactado, encharcado, erodido
Sentimentos de Risco / Alertas	preocupado, perdendo, ruim, péssimo, horror, quebra

📁 safrasmart/scores.py & recomendador.py — Algoritmo de Decisão Ponderada

O score final de 0 a 100 é calculado a partir dos seguintes pesos regulados pela solução:

- **Clima (Peso 40%):** Confronto de médias térmicas e hídricas reais contra o modelo ideal da fase fenológica.
- **Solo (Peso 20%):** Matriz de capacidade de retenção de água associada à textura minerológica.
- **Histórico (Peso 20%):** Comparação matemática da produtividade informada frente ao potencial real da CONAB.
- **Risco (Peso 20%):** Penalização direta causada por detecções textuais críticas de NLP ou extremos de calor/frio.

Matriz de Classificação das Recomendações

Métrica Numérica	Classificação de Risco	Ação / Decisão do Sistema
8.0 a 10.0	Ótimo / Seguro	Parâmetros totalmente favoráveis. Recomendação de plantio/manejo segura.
6.0 a 7.9	Bom / Regular	Condições estáveis. Recomenda-se monitoramento pontual de índices voláteis.
4.0 a 5.9	Atenção / Alerta	Presença de estresses hídricos ou térmicos moderados. Requer mitigação ativa.
0.0 a 3.9	Crítico / Impeditivo	Alto índice esperado de quebra fitotécnica. Recomendação formal de adiamento do ciclo.

📁 safrasmart/analisar.py — Orquestrador do Pipeline

Script unificador responsável por executar linearmente as 6 etapas estruturais do projeto com tratamento isolado e resiliente de exceções por etapa.

📁 safrasmart/demo.py — Execução de Casos de Uso

Módulo de simulação síncrona encarregado de injetar três cenários estáticos predefinidos para homologação rápida:

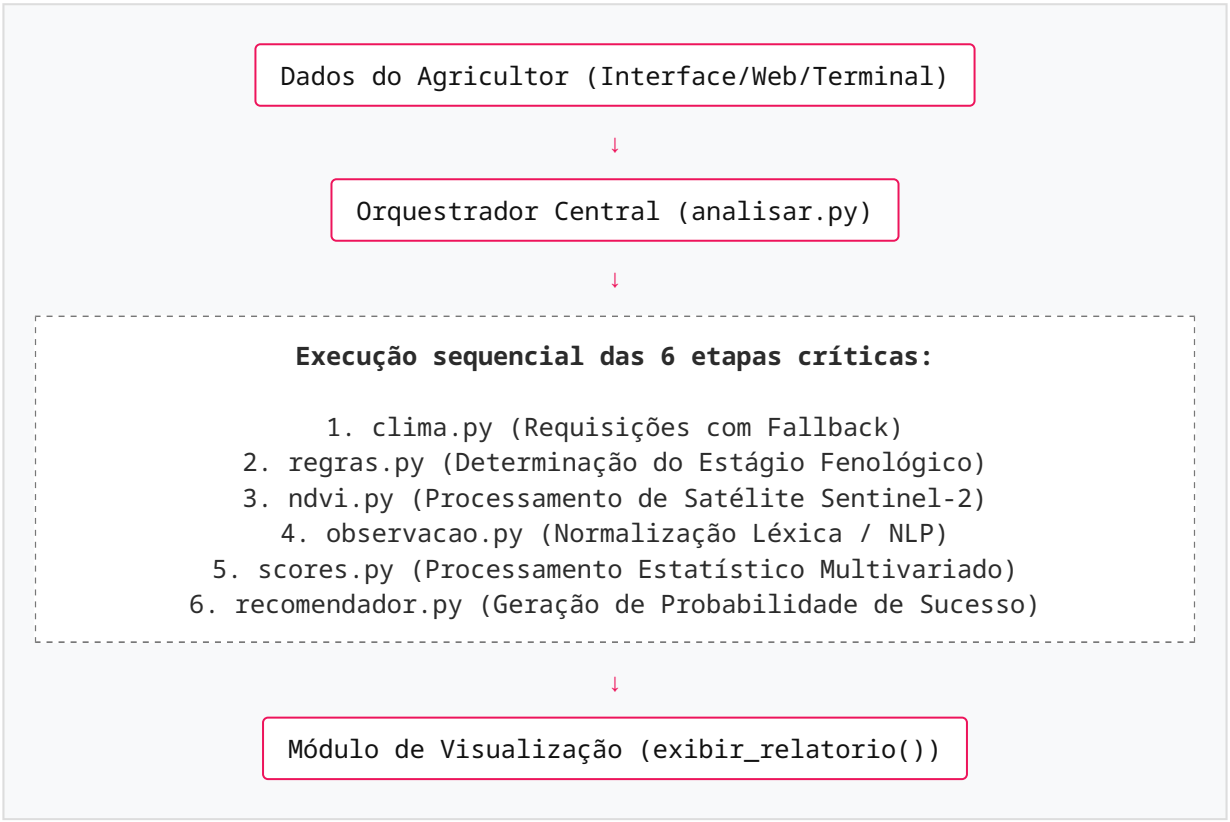
1. **Sorriso/MT:** Soja sob ótimas janelas pluviométricas (Cenário de Sucesso).
2. **Cascavel/PR:** Milho safrinha exposto a severo estresse hídrico localizado (Cenário Médio/Atenção).
3. **Uruguaiana/RS:** Soja sob excesso extremo de chuvas e risco de inundação (Cenário Crítico).

📁 safrasmart/api/server.py — Interface Web Offline-First

Disponibiliza rotinas HTTP locais (Flask) integrando formulários simplificados, captura de coordenadas por GPS nativo e buffer de persistência via `localStorage` do navegador móvel com sincronização automática (sincronização assíncrona pós-conexão móvel).

4. Fluxo Lógico e Processamento Interno (Pipeline)

O diagrama estrutural abaixo descreve como o script unificado `analisar.py` orquestra as chamadas internas síncronas sem gargalos de concorrência:


Dados do Agricultor (Interface/Web/Terminal)
↓
Orquestrador Central (analisar.py)
↓
Execução sequencial das 6 etapas críticas:
1. clima.py (Requisições com Fallback)
2. regras.py (Determinação do Estágio Fenológico)
3. ndvi.py (Processamento de Satélite Sentinel-2)
4. observacao.py (Normalização Léxica / NLP)
5. scores.py (Processamento Estatístico Multivariado)
6. recomendador.py (Geração de Probabilidade de Sucesso)
↓
Módulo de Visualização (exibir_relatorio())